

Robotic Enterprise Framework

Documentation

The Robotic Enterprise Framework is a UiPath project template for transactional automations. It provides a state machine, a configuration model, structured logging, exception handling with retries, and a test harness, so a project starts with those concerns already solved. This document describes how it is built and how to adapt it.

Item	Value
Target framework	Portable - runs on macOS, Linux and Windows
Variants	VisualBasic and CSharp - identical behaviour
Delivery	UiPath Solution (the queue is a solution resource)
Licence	MIT

Contents

1. Overview
2. Transaction Processing
3. Configuration
4. Global Constants
5. Logging
6. Exception Handling and Recovery
7. Exception Screenshots
8. Architecture and States
9. Shared Variables
10. Workflows
11. The Transaction Queue
12. Adapting the Framework
13. Expression Language
14. Test Framework
15. Prerequisites and Troubleshooting

1. Overview

A well-organised project directly affects the success of an RPA initiative. Beyond choosing processes favourable to automation and writing clear documentation, the quality of the implementation itself plays a major role. Flexible configuration, robust exception handling and meaningful logging make projects easier to implement, understand and maintain; at scale, the ability to process large volumes reliably matters too.

The REFramework covers these practices and is intended as the starting point for most RPA projects, especially those requiring scalable processing. It can be adapted to any process; its advantages are most apparent where work arrives as a stream of independent items.

2. Transaction Processing

A **transaction** is a single unit of work that can be processed independently of the others. If a process extracts data from one PDF, running it again for another file is a fresh execution; but if it is given a batch of PDFs, and each can be handled on its own, then each file is a transaction within the process.

Identifying transactions clearly is the key design decision when adopting the framework. The REFramework loops over transactions, applying the same steps - defined in the *Process Transaction* state - to each one. The source can be an Orchestrator queue (the default, and the shape this project ships wired for), a spreadsheet, a mailbox or anything else.

3. Configuration

Tunable values are **project Global Constants**, declared in `.project/globalVariables.json` and edited through Studio's Data Manager. They are typed and compiled into the project, so any workflow can reference them directly and no workflow needs a configuration argument. An expression reads `MaxRetryNumber` rather than performing a dictionary lookup and a type conversion.

To add a value, add a constant in the Data Manager and reference it by name. The full shipped set is in section 4.

Secrets do not belong here. Global Constants are compiled into the package in clear text. Store credentials, API keys and environment-specific endpoints as Orchestrator assets and read them at runtime with *Get Asset* or *Get Credential*.

4. Global Constants

The constants the framework ships with. Adjust them to the process before the first run.

Name	Type	Value	Description
LogF_BusinessProcessName	String	"REFramework_CrossPlatform"	Logging field which allows grouping of log data of two or more subprocesses under the same business process name
MaxRetryNumber	Int32	0	Must be 0 if working with Orchestrator queues. If > 0, the robot will retry the same transaction which failed with a system exception. Must be an integer value.
MaxConsecutiveSystemExceptions	Int32	0	The number of consecutive system exceptions allowed. If MaxConsecutiveSystemExceptions is reached, the job is stopped. To disable this feature, set the value to 0.
ExScreenshotsFolderPath	String	"Exceptions_Screenshots"	Where to save exceptions screenshots - can be a full or a relative path.

Name	Type	Value	Description
LogMessage_GetTransactionData	String	"Processing Transaction Number: "	Static part of logging message. Calling Get Transaction Data.
LogMessage_GetTransactionDataError	String	"Error getting transaction data for Transaction Number: "	Static part of logging message. Error retrieving Transaction Data.
LogMessage_Success	String	"Transaction Successful."	Static part of logging message. Processed Transaction successful.
LogMessage_BusinessRuleException	String	"Business rule exception."	Static part of logging message. Processed Transaction failed with business exception.
LogMessage_ApplicationException	String	"System exception."	Static part of logging message. Processed Transaction failed with application exception.
ExceptionMessage_ConsecutiveErrors	String	"The maximum number of consecutive system exceptions was reached. "	Error message in case MaxConsecutiveSystemExceptions number is reached.
RetryNumberGetTransactionItem	Int32	2	The number of times Get Transaction Item activity is retried in case of an exception. Must be an integer >= 1.
RetryNumberSetTransactionStatus	Int32	2	The number of times Set transaction status activity is retried in case of an exception. Must be an integer >= 1.
ShouldMarkJobAsFaulted	Boolean	False	Must be TRUE or FALSE. If the value is TRUE and an error occurs in Initialization state or the MaxConsecutiveSystemExceptions is reached, the job is marked as Faulted.

MaxRetryNumber deserves attention. When an Orchestrator queue is the source of transactions it should be 0, because Orchestrator manages retries itself; the retry count is then configured on the queue. Only when queues are not used should it be raised to the desired number of in-process retries. **MaxConsecutiveSystemExceptions** stops the job once that many system exceptions occur back to back; set it to 0 to disable that guard.

5. Logging

Good logging gives visibility of actions and events, makes debugging easier and produces meaningful audit trails. The framework logs transaction statuses, exceptions and state transitions at appropriate levels of the *Log Message* activity. The static parts of those messages are the LogMessage_* Global Constants.

Beyond the standard robot log fields, the framework adds custom fields with *Add Log Fields* and removes them immediately afterwards with *Remove Log Fields*, so a custom field applies only to the intended message rather than to every message that follows. The fields are logF_TransactionID, logF_TransactionField1, logF_TransactionField2, logF_TransactionNumber and logF_TransactionStatus, plus LogF_BusinessProcessName, which is added once at start-up and groups the logs of related sub-processes under one business process name.

These fields are optional but valuable for reporting. In an invoice automation, for example, the invoice number can go to the transaction ID, the date to field 1 and the total to field 2, making it possible to build volume-per-day or total-value visualisations straight from the logs.

Never log sensitive data. Logs are not encrypted.

6. Exception Handling and Recovery

The framework recovers from failures automatically, updates transaction statuses and ends execution gracefully when an exception cannot be recovered from. Exceptions fall into two categories, and the category decides whether the transaction is retried.

Category	Class	Retried?	Meaning
Business Exception	BusinessRuleException	No	The data or the situation breaks a rule of the business process - an expected attachment is missing, an amount is negative. Retrying gives the same result until a human intervenes, so the transaction is skipped and the framework fetches the next one.
System Exception	any other Exception	Yes	A technical fault - a timed-out activity, a lost connection, a crashed application. These are often transient, so the transaction is retried and the applications are re-initialised between attempts.

The two categories map onto Orchestrator's *Business Exception* and *Application Exception*. To stop a persistent fault from consuming an entire queue, `MaxConsecutiveSystemExceptions` aborts the job once that many system exceptions occur back to back.

Throw business exceptions explicitly from `Process.xaml`. Do not wrap `Process.xaml` in a catch-all Try Catch - the framework's own handler is what classifies and routes the exception. Do not add a Global Exception Handler either; it conflicts with the per-state Try Catch.

7. Exception Screenshots

When a transaction fails with a **system exception**, `Framework/SetTransactionStatus.xaml` invokes `Framework/TakeScreenshot.xaml`, which captures the whole desktop and writes a PNG into the folder named by the `ExScreenshotsFolderPath` Global Constant (`Exceptions_Screenshots` by default). Successful transactions and business rule exceptions produce no screenshot.

The file name is the path passed in, with a timestamp inserted before the extension - for example `Exceptions_Screenshots/ExceptionScreenshot_2026-08-28_112451_186.png`. The coded workflow returns the path it actually wrote; `TakeScreenshot.xaml` assigns that back to `io_FilePath` and logs Screenshot saved at: , and `SetTransactionStatus.xaml` records the same path in the queue item's `Details` field, so the screenshot can be found from Orchestrator.

This feature is most useful for unattended runs, where it provides evidence of the screen state at the moment of failure with nobody watching.

8. Architecture and States

`Main.xaml` is a state machine with four states.

State	Behaviour
Initialization	Kill leftover processes and initialise the applications used by the process. On success the execution moves to Get Transaction Data; on failure, to End Process. After a system exception the framework closes everything and returns here, so applications are re-initialised before the retry.
Get Transaction Data	Retrieve the next transaction. If there is none, or an error occurs, move to End Process. Otherwise move to Process Transaction.
Process Transaction	Process one transaction. The outcome is Success, Business Exception or System Exception. A system exception may be retried; a business exception skips the transaction and the framework fetches the next one.
End Process	Close all applications and finish. Reached on normal completion and on abort alike.

Workflows invoked per state

State	Invoked workflows
Initialization	KillAllProcesses.xaml, InitAllApplications.xaml
Get Transaction Data	GetTransactionData.xaml
Process Transaction	Process.xaml, SetTransactionStatus.xaml (which may in turn invoke RetryCurrentTransaction.xaml, TakeScreenshot.xaml, CloseAllApplications.xaml and KillAllProcesses.xaml)
End Process	CloseAllApplications.xaml, KillAllProcesses.xaml

9. Shared Variables

The variables declared in Main.xaml and passed to the invoked workflows.

Name	Type	Description
TransactionItem	QueueItem	The transaction being processed. Change this type to match the process - DataRow for spreadsheet rows, MailMessage for emails, String for file paths.
SystemException	Exception	Carries a non-business exception between states.
BusinessException	BusinessRuleException	Carries a business-rule violation between states.
TransactionNumber	Int32	Sequential counter of transaction items. Starts at 1.
IsFirstRun	Boolean	True until Initialization has run once, so the first-run block is skipped when the state re-runs after a system exception.
RetryNumber	Int32	Attempts made at retrying the current transaction after a system exception.
TransactionField1	String	Optional extra information about the transaction, for logging.
TransactionField2	String	Optional extra information about the transaction, for logging.
TransactionID	String	Identifier used for logging. Ideally unique per transaction.
dt_TransactionData	DataTable	Holds transactions when they come from a table rather than a queue.
ConsecutiveSystemExceptions	Int32	Consecutive system exceptions so far; compared against MaxConsecutiveSystemExceptions.

10. Workflows

Each workflow, its purpose and its arguments. Configuration reaches every workflow as Global Constants, so none of them take a configuration argument.

Framework/KillAllProcesses.xaml

Forcibly terminates processes left running from a previous execution. Ships empty apart from a log message; add *Kill Process* activities for the applications your process uses. Takes no arguments.

Framework/InitAllApplications.xaml

Opens and authenticates the applications the process operates. Add *Use Application/Browser* activities here, or invoke workflows that perform login. Takes no arguments; read any URLs or paths it needs from Global Constants or Orchestrator assets.

Framework/GetTransactionData.xaml

Retrieves the next transaction. By default the first activity is *Get Transaction Item*, pointing at the queue described in section 11. Returning Nothing from out_TransactionItem ends the process.

Argument	Direction	Description
in_TransactionNumber	In	Index of the transaction to fetch. Use it when the source is not a queue.
out_TransactionItem	Out	The transaction to process, or Nothing when there are none left.
out_TransactionID	Out	Identifier for logging.
out_TransactionField1	Out	Optional extra information for logging.
out_TransactionField2	Out	Optional extra information for logging.
io_dt_TransactionData	In/Out	Table of transactions, when the source is tabular.

Framework/Process.xaml

Your business logic - the part of the framework you are expected to write. Runs once per transaction.

Argument	Direction	Description
in_TransactionItem	In	The transaction to process. With a queue, fields are read as in_TransactionItem.SpecificContent("FieldName").

Framework/SetTransactionStatus.xaml

Sets and logs the outcome of each transaction - Success, Business Exception or System Exception - and updates the queue item in Orchestrator accordingly. On a system exception it invokes RetryCurrentTransaction.xaml and TakeScreenshot.xaml, and re-initialises the applications through CloseAllApplications.xaml and KillAllProcesses.xaml.

Argument	Direction	Description
in_TransactionItem	In	The transaction whose status is being set.
in_SystemException	In	Set when the transaction failed with a system exception.
in_BusinessException	In	Set when the transaction failed with a business exception.
in_TransactionID	In	Identifier for logging.
in_TransactionField1	In	Optional extra information for logging.
in_TransactionField2	In	Optional extra information for logging.
io_RetryNumber	In/Out	Attempts made at the current transaction.
io_TransactionNumber	In/Out	Sequential counter, advanced when moving to the next transaction.
io_ConsecutiveSystemExceptions	In/Out	Running count of consecutive system exceptions.

Framework/RetryCurrentTransaction.xaml

Implements the retry mechanism, invoked from SetTransactionStatus.xaml when a system exception occurs. When the queue manages retries it simply advances; otherwise it compares io_RetryNumber against the MaxRetryNumber Global Constant and decides whether to retry the same transaction or move on.

Argument	Direction	Description
in_SystemException	In	The exception that triggered the retry.

Argument	Direction	Description
in_QueueRetry	In	True when retries are managed by an Orchestrator queue.
io_RetryNumber	In/Out	Attempts made at the current transaction.
io_TransactionNumber	In/Out	Sequential counter of transactions.

Framework/TakeScreenshot.xaml

Computes the screenshot path, creates the folder if needed, and invokes the coded workflow that captures the desktop (section 7).

Argument	Direction	Description
in_Folder	In	Folder the screenshot is written to.
io_FilePath	In/Out	Optional path in; on the way out, the path actually written, including the timestamp.

Framework/CloseAllApplications.xaml

Closes the applications used by the process gracefully - logging out where appropriate. Runs on every exit path, successful or not, and also between retries. Takes no arguments.

NTakeScreenshot_Coded.cs

A coded workflow that captures the desktop through the UiAutomation coded API and writes it as a PNG. Takes `in_ScreenshotPath` and an optional `in_TimeoutMS`, and returns the path it wrote, timestamp included.

11. The Transaction Queue

The queue is declared as the solution resource **TransactionQueue** (`resources/solution_folder/queue/TransactionQueue.json`). The solution owns its identity and provisions it at deploy time.

- **Get Transaction Item** in `Framework/GetTransactionData.xaml` refers to the queue by that name. Rename the resource and the activity together.
- **The folder is deliberately left unset**, so the robot's own Orchestrator folder applies. Deploy the process into the folder that holds the queue.
- Queue activities require a folder in scope. Running one outside a folder returns 400 - A folder is required for this action. Error code: 1101, which reads like an authentication failure but is not. Error code: 1002 means the named queue does not exist in that folder.

12. Adapting the Framework

Before the first run:

- **Set the Global Constants** (section 4). At minimum set `LogF_BusinessProcessName` to the name of your process, since it is stamped on every log message the framework emits.
- **Point the queue resource at your queue** (section 11), or replace the *Get Transaction Item* activity in `GetTransactionData.xaml` if the source is not a queue.
- **Implement `InitAllApplications.xaml` and `CloseAllApplications.xaml`** for the applications your process drives, and add the matching *Kill Process* activities to `KillAllProcesses.xaml`.

- **Implement Process.xaml.** Throw `BusinessRuleException` for data problems and let everything else propagate as a system exception.
- **Set the transaction type.** The default is `QueueItem`. If you change the type of the `TransactionItem` variable in `Main.xaml` - to `DataRow` for spreadsheet rows, for example - change it in `GetTransactionData.xaml`, `Process.xaml` and `SetTransactionStatus.xaml` too, then use *Import Arguments* on each Invoke Workflow File so the bindings match. Updating one file and not the others surfaces as argument-mismatch errors at design time.
- **Keep MaxRetryNumber at 0 while developing** so failures surface immediately, and set the production value before deployment.

13. Expression Language

The framework is available in a VisualBasic and a CSharp variant, identical in behaviour. The expression language is fixed when a project is created and applies to every workflow in it, so pick the variant that matches your team before starting.

In the CSharp variant three things differ in the way expressions are written:

- Global Constants are referenced qualified, as `GlobalConstants.<Name>`.
- Names are case-sensitive, so they must match the declaration exactly.
- Every expression lives in a `<CSharpValue>` or `<CSharpReference>` element rather than in an attribute.

14. Test Framework

The project ships with test cases that exercise the framework's components and let you verify each exit path before deploying.

File	Purpose
Tests/MainTestCase.xaml	Runs every workflow listed on the Tests sheet of Tests.xlsx, classifies each outcome as Success, BusinessRuleException or SystemException, and writes the Status (PASS/FAIL) and Comments back to the Result sheet. It then asserts that no row failed. Only workflows that take no arguments can be listed, because the loop invokes them without passing any.
Tests/GetTransactionDataTestCase.xaml	Verifies that a transaction item is retrieved. Needs a reachable queue and an Orchestrator folder.
Tests/ProcessTestCase.xaml	Exercises Process.xaml against a retrieved transaction. Same prerequisites.
Tests/InitAllApplicationsTestCase.xaml	Verifies application initialisation and clean shutdown.
Tests/WorkflowTestCaseTemplate.xaml	A blank Given/When/Then template for your own test cases.
Tests/Tests.xlsx	Two sheets. Tests lists the workflows to run and the result expected from each; Result receives the outcome of the last run.

Excel is read and written with the Workbook activities, which use a file-based engine and need no Excel installation. Tests/MainTestCase.xaml is the only Excel consumer in the project. Treat the cases as samples - extend them for your process, and put a CI/CD pipeline in place.

15. Prerequisites and Troubleshooting

Topic	Detail
.NET SDK 8.0	Must be on PATH - the workflow compiler requires it by name. A newer SDK may be installed alongside it.
Licensing	A Studio licence must be assigned to the signed-in user on the active organization, otherwise validate, build and run fail with a feature-not-available error.
Editing Global Constants outside Studio	The compiled globals assembly is cached per Studio host session and outlives the source change. After hand-editing globalVariables.json, delete .local/.jit and .local/install and restart the Studio host, or newly added names will not resolve. The same fix applies to a CS1705 about System.Private.CoreLib versions.
Running from the CLI	uip rpa run writes .local/.codedworkflows/CodedWorkflow.cs but not the WorkflowRunnerService it references, so runs after the first fail with "WorkflowRunnerService does not exist". Clear .local/.codedworkflows, .local/.jit and .local/install before each CLI run. Studio is unaffected.
Error code 1101 / 1002	A queue activity ran without a folder in scope, or the named queue does not exist in that folder. See section 11.

16. Distribution

The REFramework is available under the MIT License and distributed as a UiPath Studio template and via <https://github.com/UiPath-Services/StudioTemplates>. Adapting it to particular use cases and transaction types is expected and encouraged - the framework is a starting point, not a fixed contract.